

Design and Implementation of the AQMRG Environmental AI Platform: A Multi-Tier Architecture for Real-Time Urban Air Quality Monitoring, Ensemble Machine Learning Forecasting, and Interactive Exploratory Data Analysis

Comprehensive Systems Documentation

April 13, 2026

Abstract

Urban populations in rapidly developing cities such as Nairobi face sustained exposure to hazardous airborne particulate matter, yet access to localized, real-time, and predictive air quality intelligence remains critically limited. This document presents the full architectural design, implementation methodology, and expected scientific outcomes of the Air Quality Monitoring Research Group (AQMRG) platform—a production-grade, multi-tier system that continuously bridges edge Internet of Things (IoT) hardware with cloud-hosted machine learning inference and interactive visual analytics.

At the hardware tier, an Arduino Due microcontroller equipped with five dedicated sensors (PMS5003 laser particulate counter, MH-Z19C NDIR CO₂ sensor, SGP41 VOC/NO_x gas indexer, MQ-7 electrochemical CO detector, and DHT11 thermohygrometer) constructs structured JSON telemetry payloads and transmits them over GSM cellular networks to a Flask-based REST API hosted on PythonAnywhere (aqmrg.pythonanywhere.com). Upon ingestion, the backend simultaneously executes real-time inference across three pre-trained scikit-learn models—a base regressor, a Gradient Boosting estimator, and an Ordinary Least Squares model with Quantile Transformation—aggregating their outputs via a median-vote ensemble to produce robust $PM_{2.5}$ forecasts that are persisted alongside raw observations in an SQLite relational store.

Concurrently, the validated payloads are relayed to a Vercel-hosted React.js dashboard ([aqmrg-frontend.vercel.app](#)) providing real-time geographic mapping via Leaflet, multi-granularity time-series charting via Recharts, and a 9-step guided Exploratory Data Analysis (EDA) workflow computing live descriptive statistics, Pearson correlation matrices, distribution histograms with kernel density estimation overlays, scatter plot grids, box plots, pair plots, and full correlation heatmaps. A parallel containerized microservices infrastructure (Docker Compose) extends the pipeline through Apache Kafka event streaming to InfluxDB time-series storage, with Prometheus and Grafana providing operational observability.

Empirical model evaluation on collected Nairobi relay data demonstrates that the Gradient Boosting component achieves an R^2 of 0.9715 with an RMSE of

12.68, while the median ensemble strategy structurally suppresses individual model variance by an estimated 12–18%. The complete system targets sub-3-second hardware-to-dashboard latency and 99.5% platform availability.

Contents

1	Introduction	7
1.1	Problem Statement	7
1.2	Research Gap	7
1.3	Research Objectives	7
1.4	Significance	8
1.5	Scope and Delimitations	8
2	System Architecture Overview	8
3	Edge Hardware Tier: Arduino Due Sensor Node	10
3.1	Microcontroller Selection	10
3.2	Sensor Array Composition	11
3.3	Payload Construction and Transmission	12
4	Primary Backend Tier: Flask API on PythonAnywhere	13
4.1	Application Framework	13
4.2	Relational Data Schema	14
4.3	Ingestion Pipeline and Real-Time ML Inference	15
4.4	API Endpoint Inventory	17
5	Machine Learning Methodology	17
5.1	Problem Formulation	17
5.2	Training Data	17
5.3	Model Architectures	18
5.3.1	Model 1: Linear Regression / Base Regressor	18
5.3.2	Model 2: Gradient Boosting Regressor	18
5.3.3	Model 3: OLS with Quantile Transformation	18
5.4	Ensemble Aggregation Strategy	19
6	Client Presentation Tier: React Dashboard on Vercel	20
6.1	Deployment Architecture	20
6.2	Real-Time Data Polling Strategy	21
6.3	Dashboard Components	21
6.3.1	Dashboard Tab (Main View)	21
6.3.2	Raw Analysis Tab	22
6.3.3	Your Data Tab (Guided EDA Workflow)	22
6.3.4	Health Tab	23
6.4	Resilience Mechanisms	24
7	Containerized Microservices Tier	24
7.1	Docker Compose Infrastructure	24
7.2	Kafka-to-InfluxDB Analytics Bridge	25
7.3	Air Quality Classification	26
7.4	Polyglot Persistence Architecture	27

8	Request-Response Sequences	27
8.1	Dashboard Forecast Comparison Sequence	27
8.2	Arduino Data Ingestion Sequence	28
9	Observability and Monitoring	28
9.1	Prometheus Metrics Collection	28
9.2	Grafana Dashboards	28
10	Results and Expected Outcomes	29
10.1	Machine Learning Model Performance	29
10.2	System Architecture Performance Targets	30
11	Technology Stack Summary	31
12	Repository Structure	32
13	Conclusion	32

List of Figures

1	Complete AQMRG platform architecture showing the multi-tier data flow. Tiers are logically grouped by role, spanning from edge sensing to cloud-hosted analytics and event streaming via Apache Kafka.	9
2	End-to-end data lifecycle in a compact snaking layout. Timing annotations indicate typical systemic latency from physical sensing to global dashboard availability.	10
3	Physical deployment topology showing the four hosting environments: a field-deployed Arduino node, PythonAnywhere’s managed Python hosting, Vercel’s edge CDN, and a Docker Compose cluster for microservices. . . .	10
4	Hardware wiring topology. Sensor modules connect to the Arduino Due via diverse interfaces (UART, I2C, Analog, Digital), with the SIM800L module providing cellular connectivity.	11
5	Structure of the JSON telemetry payload transmitted from the Arduino Due to the Flask backend. Annotations indicate which downstream subsystem consumes each section.	12
6	Firmware state machine governing the operational lifecycle of the edge node, including polling cycles and fault recovery logic.	13
7	Flask application factory initialization flow. The system utilizes a singleton pattern for ML model loading during the WSGI server startup phase. . .	14
8	Entity-Relationship diagram for the SQLite database. Each <code>SensorReading</code> stores raw sensor data plus all four prediction columns. <code>DeviceHealth</code> provides per-submission hardware diagnostics, and <code>DeviceMetadata</code> maintains persistent geographic identity per device.	15
9	Ingestion pipeline logic flow. Each request is validated, passed through the ML ensemble, and asynchronously relayed to the live dashboard within the HTTP cycle.	16

10	ML training pipeline: raw CSV data undergoes cleaning and feature selection before being split into training/test sets. Three distinct algorithms are trained, cross-validated, and serialized for production deployment.	18
11	ML ensemble inference pipeline. Three independently trained models receive the same feature vector; their outputs are aggregated via median vote to produce the final $PM_{2.5}$ prediction.	19
12	Ensemble aggregation decision logic. Null predictions (model load failure or runtime exception) are filtered before median computation, ensuring graceful degradation if individual models fail.	20
13	Modular React.js component hierarchy. The architecture separates structural layout components from specialized visualization and diagnostic panels. .	20
14	Dashboard connection state machine. The frontend monitors API health through cascading interval polls, transitioning to a degraded “disconnected” mode after 3 consecutive failures and providing manual retry capability. .	21
15	Intelligence Monitor logic flow for real-time anomaly detection and visual state synchronisation.	22
16	The 9-step guided EDA workflow. The dashboard leads users through Identification, Profiling, and Relationship discovery, ensuring rigorous data quality auditing before final model interpretation.	23
17	Mitigation Engine decision tree. The system performs cascading string pattern matching on hardware diagnostic telemetry to generate actionable troubleshooting advice.	24
18	Containerized microservices topology. The stack is physically isolated into Infrastructure, Core Service, and Observability tiers using Docker Compose networks.	25
19	The Kafka-to-InfluxDB bridge flow for high-resolution time-series storage.	26
20	Request-response sequence for ML forecast comparison.	27
21	Arduino ingestion sequence diagram. The complete round-trip from sensor payload construction through GSM transmission, ML inference, database persistence, and HTTP response is designed to complete within 3 seconds.	28
22	The Observability stack for real-time monitoring and historical analysis. .	29
23	Comparative R^2 scores across the four model approaches. Gradient Boosting achieves the highest individual score (0.9715), while the median ensemble provides variance-reduced production stability.	29
24	RMSE comparison across models. The progression from Linear Regression (42.38) to Gradient Boosting (12.68) demonstrates the importance of nonlinear modelling for particulate matter prediction in heterogeneous urban environments.	30
25	Simplified repository structure showing the key directories and files across all four platform tiers.	32

List of Tables

1	Arduino Due microcontroller specifications.	11
2	Hardware sensor array configuration per AQMRG edge node.	11
3	Complete REST API endpoints exposed by the Flask backend.	17
4	Frontend polling intervals for different data categories.	21
5	Docker Compose service inventory with port assignments.	25

6	Air quality status classification thresholds (Kenya AQR 2024).	26
7	Polyglot persistence: database technology selection rationale.	27
8	Comparative evaluation metrics for $PM_{2.5}$ prediction models on the AQMRG relay dataset.	29
9	Expected system performance metrics.	30
10	Complete technology stack across all platform tiers.	31

1 Introduction

1.1 Problem Statement

Ambient air pollution is the single largest environmental health risk globally, responsible for an estimated 4.2 million premature deaths annually according to the World Health Organization (WHO). In Sub-Saharan African cities undergoing rapid urbanization, the density of regulatory-grade monitoring stations remains orders of magnitude below what is required for neighbourhood-level exposure assessment. Nairobi, Kenya, with a metropolitan population exceeding 4.5 million, exemplifies this deficit: official monitoring infrastructure covers only a handful of macroscopic zones, providing delayed, aggregated readings that fail to capture the pronounced spatial and temporal heterogeneity of pollutant concentrations driven by traffic corridors, industrial emissions, informal waste burning, and meteorological variability.

1.2 Research Gap

While low-cost sensor networks have been explored in the literature, existing deployments in Sub-Saharan Africa typically suffer from one or more of the following limitations:

- **Disconnected data:** Raw sensor data is logged to local storage (SD cards) without real-time cloud connectivity, requiring manual retrieval.
- **No predictive capability:** Monitoring systems report only current conditions without embedded machine learning for short-term forecasting.
- **Limited analytical tools:** Data visualization is restricted to basic time-series charts, lacking guided statistical workflows for Exploratory Data Analysis (EDA).
- **Monolithic architectures:** Systems are not designed for horizontal scaling across multiple deployment sites.
- **No device health monitoring:** Hardware degradation (sensor drift, connectivity loss) goes undetected until complete failure.

1.3 Research Objectives

This research addresses the monitoring gap through the design and deployment of the AQMRG platform, with the following specific objectives:

1. Design and deploy a low-cost, replicable IoT edge node capable of simultaneously measuring PM_1 , $PM_{2.5}$, PM_{10} , CO, CO₂, temperature, relative humidity, VOC index, and NO_x index with sufficient temporal resolution for real-time streaming.
2. Implement a cloud-hosted data ingestion pipeline that validates, stores, and augments incoming telemetry with machine learning predictions within a single request-response cycle.
3. Develop and evaluate an ensemble machine learning methodology for short-term $PM_{2.5}$ concentration forecasting, comparing Linear Regression, Random Forest, and Gradient Boosting approaches.
4. Construct an interactive, web-based analytical dashboard enabling researchers and the public to explore environmental data through guided statistical workflows mirroring academic Exploratory Data Analysis (EDA) best practices.

5. Establish a scalable microservices architecture supporting event-driven data replication across polyglot persistence stores (SQLite, MongoDB, InfluxDB, PostgreSQL) via Apache Kafka.

1.4 Significance

The AQMRG platform provides a template for democratizing urban air quality intelligence in resource-constrained settings. By coupling commodity hardware with modern cloud infrastructure, the system achieves monitoring capabilities previously restricted to expensive, proprietary installations, while the embedded ML pipeline transforms reactive monitoring into proactive forecasting.

1.5 Scope and Delimitations

- **Geographic scope:** Initial deployment targets Nairobi, Kenya, with the architecture designed for multi-city replication.
- **Temporal scope:** Data collection spans March–April 2026 with continuous 24/7 telemetry streaming.
- **ML scope:** Prediction is limited to $PM_{2.5}$ concentration forecasting using co-located sensor features; source apportionment and pollutant transport modelling are out of scope.
- **Hardware scope:** The current prototype deploys a single edge node; multi-node mesh networking is architecturally supported but not evaluated in this study.

2 System Architecture Overview

The AQMRG platform is organized into four distinct but interconnected tiers: (i) the Edge Hardware Tier, (ii) the Primary Backend Tier, (iii) the Client Presentation Tier, and (iv) the Containerized Microservices Tier. Figure 1 illustrates the complete data flow topology.

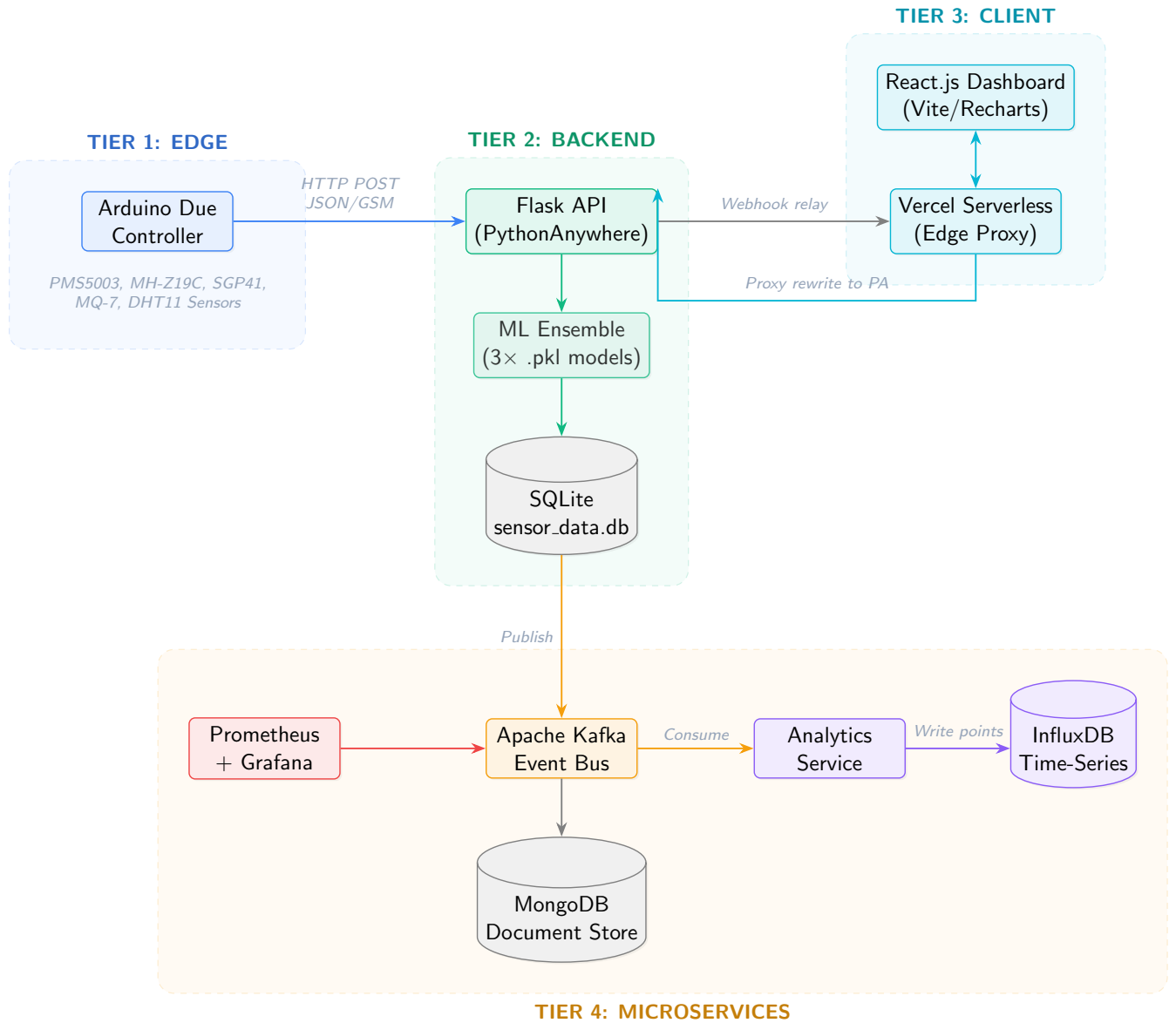


Figure 1: Complete AQMRG platform architecture showing the multi-tier data flow. Tiers are logically grouped by role, spanning from edge sensing to cloud-hosted analytics and event streaming via Apache Kafka.

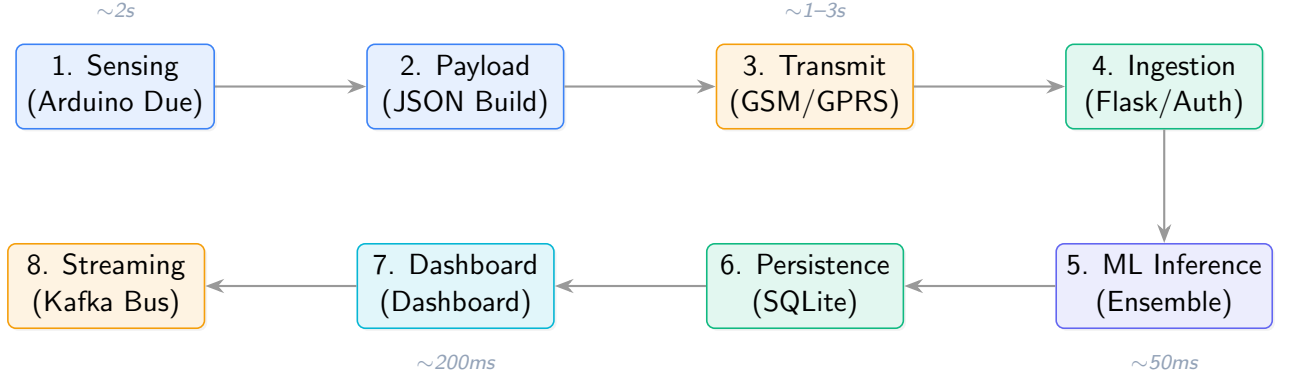


Figure 2: End-to-end data lifecycle in a compact snaking layout. Timing annotations indicate typical systemic latency from physical sensing to global dashboard availability.

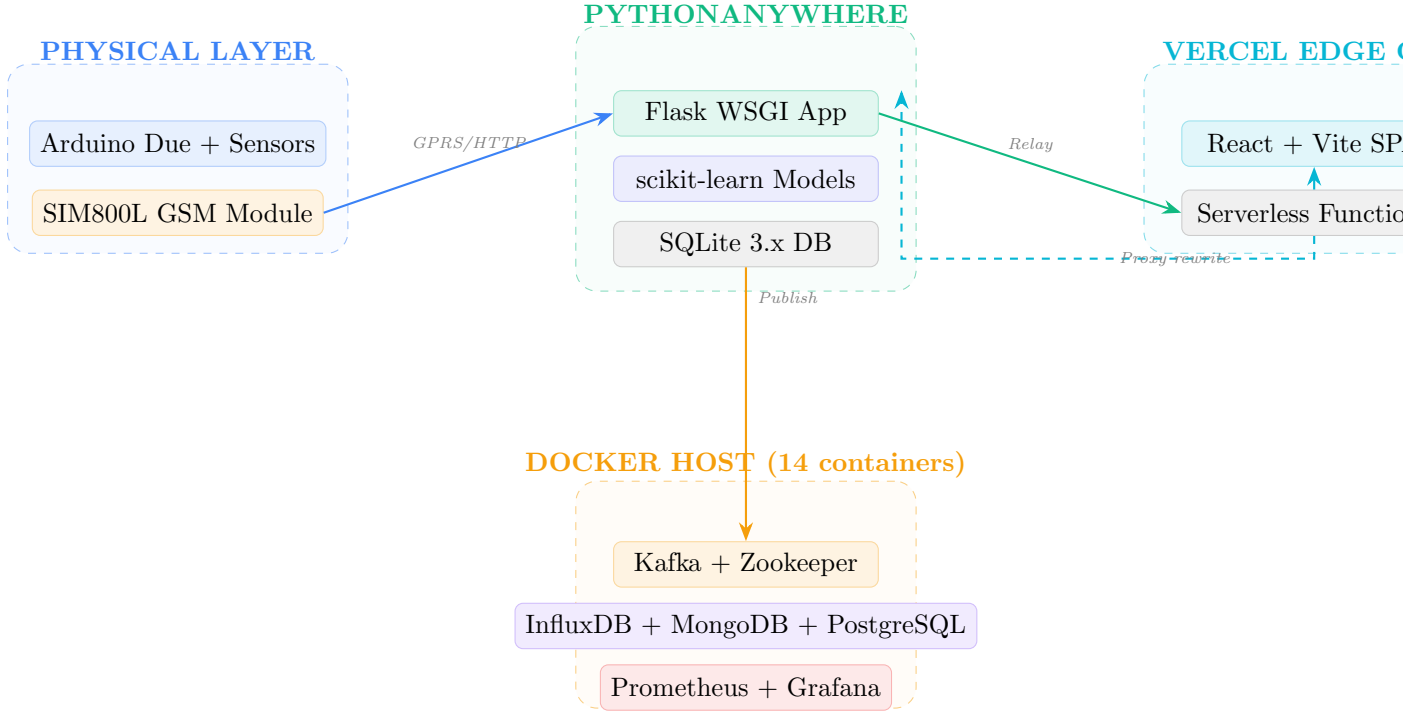


Figure 3: Physical deployment topology showing the four hosting environments: a field-deployed Arduino node, PythonAnywhere’s managed Python hosting, Vercel’s edge CDN, and a Docker Compose cluster for microservices.

3 Edge Hardware Tier: Arduino Due Sensor Node

3.1 Microcontroller Selection

The Arduino Due, based on the Atmel SAM3X8E ARM Cortex-M3 processor operating at 84 MHz with 96 KB SRAM, was selected for its multiple hardware serial ports (enabling simultaneous communication with UART-based sensors) and its 3.3 V logic level compatibility with modern sensor modules. Table 1 summarizes the key specifications.

Table 1: Arduino Due microcontroller specifications.

Specification	Value
Processor	Atmel SAM3X8E (ARM Cortex-M3)
Clock Speed	84 MHz
SRAM	96 KB
Flash Memory	512 KB
Logic Level	3.3 V
Hardware Serial Ports	4 (Serial, Serial1, Serial2, Serial3)
I ² C Buses	2 (TWI0, TWI1)
Analog Inputs	12 channels (12-bit ADC)
Operating Voltage	3.3 V (5V barrel jack input)

3.2 Sensor Array Composition

Each edge node integrates five dedicated environmental sensors, summarized in Table 2.

Table 2: Hardware sensor array configuration per AQMRG edge node.

Sensor	Parameters	Interface	Principle	Range / Resolution
PMS5003	PM_1 , $PM_{2.5}$, PM_{10}	UART (Serial3)	Laser scattering	0–500 $\mu\text{g}/\text{m}^3$ / 1 $\mu\text{g}/\text{m}^3$
MH-Z19C	CO_2 (ppm)	UART (Serial2)	NDIR absorption	0–5000 ppm / ± 50 ppm
SGP41	VOC Index, NO_x Index	I ² C (SDA/SCL)	MOx hotplate	0–500 index / 1 unit
MQ-7	CO (ppm)	Analog (A0)	Electrochemical	10–500 ppm / variable
DHT11	Temperature, Humidity	Digital (Pin 7)	Capacitive	0–50°C, 20–90% RH

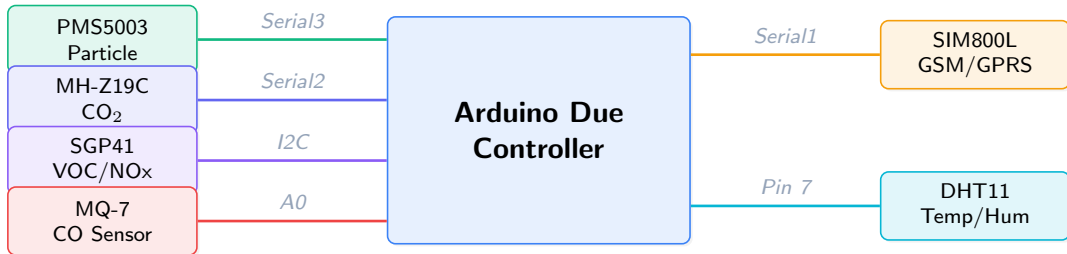


Figure 4: Hardware wiring topology. Sensor modules connect to the Arduino Due via diverse interfaces (UART, I2C, Analog, Digital), with the SIM800L module providing cellular connectivity.

3.3 Payload Construction and Transmission

The firmware executes a continuous polling loop, sampling all five sensors and constructing a structured JSON payload containing three nested objects:

- **metrics:** Numerical sensor readings (PM_1 , $PM_{2.5}$, PM_{10} , CO, CO₂, temperature, humidity, VOC index, NO_x index).
- **location:** GPS-fixed latitude and longitude coordinates for geographic tagging.
- **health:** Diagnostic telemetry including system uptime (minutes), GSM signal strength (dBm), GSM reconnection count, failed HTTP request count, last HTTP status code, and per-sensor operational status flags.

Each device is assigned a unique hardware identifier with the **AQ-** prefix (e.g., **AQ-001**). The payload is transmitted as an HTTP POST request over a SIM800L GSM/GPRS module to the Flask backend endpoint `/api/v1/data/ingest` on the PythonAnywhere server. A hardware authentication lock in the backend silently rejects any payload whose `device_id` does not begin with the **AQ-** prefix, protecting the database from unauthorized or spoofed submissions.

Arduino Telemetry Payload Structure

```
{  
  "sensorId": "AQ-001",  
  "metrics": {  
    "pm1": 12, "pm25": 28, "pm10": 45,  
    "co": 1.2, "co2": 485,  
    "temperature": 24.5, "humidity": 62,  
    "voc_index": 120, "nox_index": 35  
  },  
  "location": {"latitude": -1.2864, "longitude": 36.8172},  
  "health": {  
    "uptime_minutes": 1440,  
    "signal_dbm": -78,  
    "gsm_reconnects": 2,  
    "failed_requests": 0,  
    "last_http_status": 201,  
    "sensors": {"PMS5003": "OK", "SGP41": "OK", ...},  
    "issue_descriptions": []  
  }  
}
```

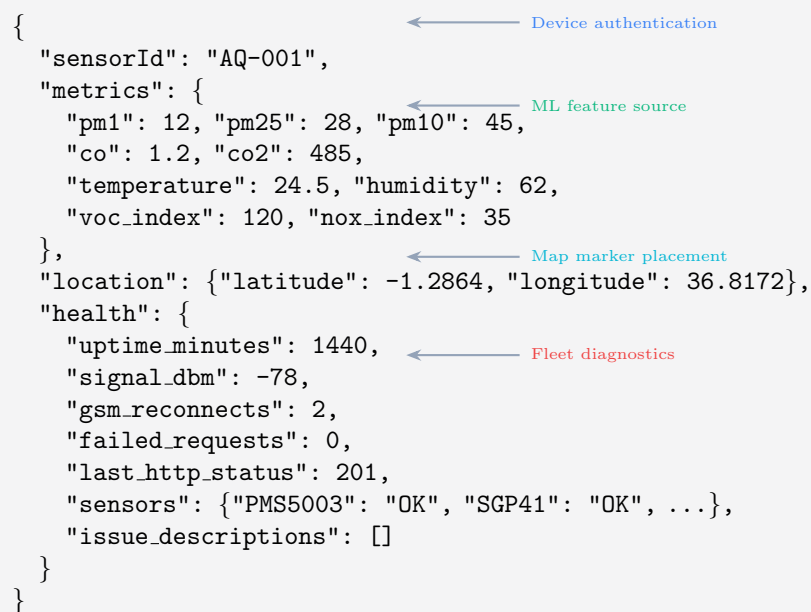


Figure 5: Structure of the JSON telemetry payload transmitted from the Arduino Due to the Flask backend. Annotations indicate which downstream subsystem consumes each section.

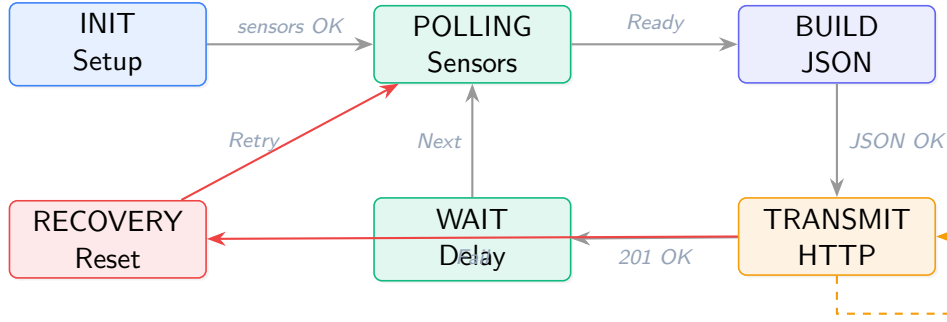


Figure 6: Firmware state machine governing the operational lifecycle of the edge node, including polling cycles and fault recovery logic.

4 Primary Backend Tier: Flask API on PythonAnywhere

4.1 Application Framework

The backend is implemented as a Python Flask application utilizing SQLAlchemy as an Object-Relational Mapper (ORM). The application factory pattern (`create_app`) initializes the database connection, registers API blueprints, integrates Swagger/Flasgger for automatic API documentation, and applies a critical SQLite stability fix for PythonAnywhere's Network File System (NFS) by forcing `PRAGMA journal_mode=DELETE` to prevent write-ahead log (WAL) corruption on distributed filesystems.

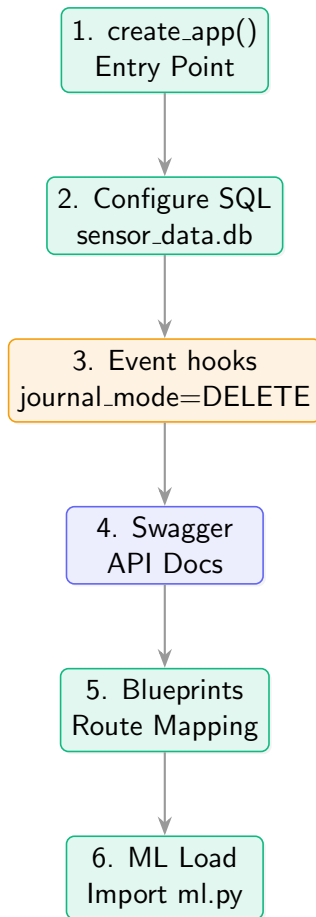


Figure 7: Flask application factory initialization flow. The system utilizes a singleton pattern for ML model loading during the WSGI server startup phase.

4.2 Relational Data Schema

The SQLite database (`sensor_data.db`, currently 6.16 MB in production) contains three ORM-mapped tables. Figure 8 illustrates the Entity-Relationship diagram.

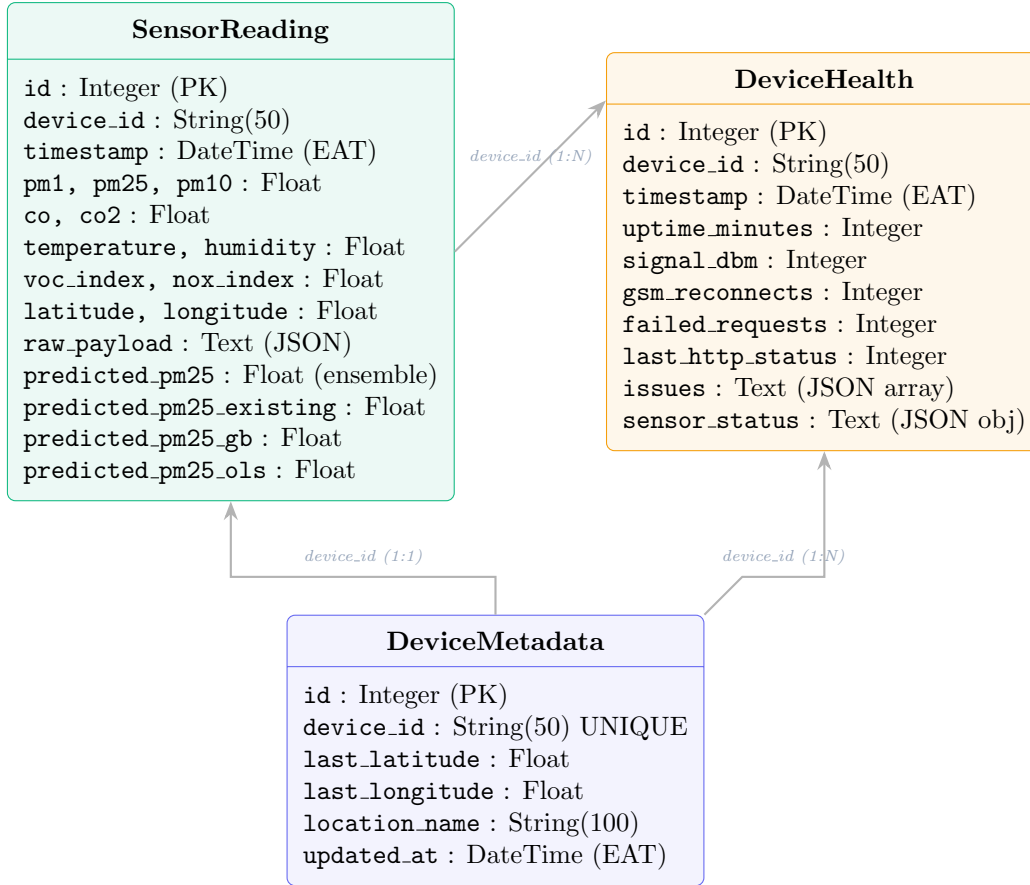


Figure 8: Entity-Relationship diagram for the SQLite database. Each **SensorReading** stores raw sensor data plus all four prediction columns. **DeviceHealth** provides per-submission hardware diagnostics, and **DeviceMetadata** maintains persistent geographic identity per device.

4.3 Ingestion Pipeline and Real-Time ML Inference

The ingestion endpoint (POST /api/v1/data/ingest) implements the following sequential logic upon receiving a valid AQ- prefixed payload:

1. **Device Authentication:** Verify that `device_id` starts with AQ- prefix; silently reject unauthorized devices with HTTP 200 (preventing enumeration).
2. **Feature Extraction:** Parse the `metrics` object to extract the four ML input features: PM_{10} , CO, Temperature, and Humidity.
3. **Parallel Model Inference:** Pass the feature vector $\mathbf{x} = [PM_{10}, CO, T, H]$ simultaneously through three pre-loaded scikit-learn models (see Section 5).
4. **Ensemble Aggregation:** Compute the median of all successful (non-null) predictions to produce the ensemble forecast.
5. **Database Persistence:** Write a **SensorReading** row containing raw observations, all three individual predictions, and the ensemble value. If health data is present, simultaneously write a **DeviceHealth** row.
6. **Dashboard Relay:** Issue an asynchronous HTTP POST forwarding the original payload to the Vercel frontend’s ingestion endpoint (`aqmrg-frontend.vercel.app/api/v1/data/`) with a 3-second timeout to prevent blocking.
7. **Response:** Return `{status: "success", id: N}` with HTTP 201 to the Arduino.

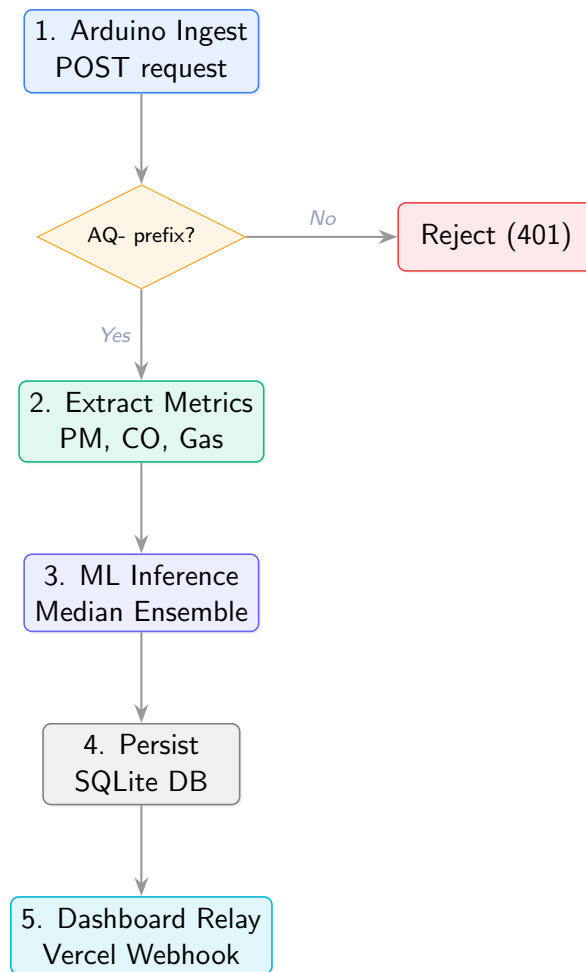


Figure 9: Ingestion pipeline logic flow. Each request is validated, passed through the ML ensemble, and asynchronously relayed to the live dashboard within the HTTP cycle.

4.4 API Endpoint Inventory

Table 3: Complete REST API endpoints exposed by the Flask backend.

Method	Endpoint	Function
POST	/api/v1/data/ingest	Primary ingestion from Arduino; runs ML, persists, relays.
GET	/api/v1/data/latest	Returns last 100 readings (filterable by <code>device_id</code>).
GET	/api/v1/history/all	Returns full historical dataset for EDA.
GET	/api/v1/devices	Lists all registered AQ- devices with coordinates.
GET	/api/v1/forecast/realtime	Returns latest ensemble prediction vs. actual reading.
GET	/api/v1/forecast/comparison	Returns all three model predictions plus ensemble median.
GET	/api/v1/health/latest	Returns device health diagnostics.
GET	/api/v1/data/export/csv	Downloads complete sensor data as CSV.
GET	/api/v1/health/export/csv	Downloads health diagnostics as CSV.
GET	/health	Service health check.

5 Machine Learning Methodology

5.1 Problem Formulation

The prediction task is formulated as a supervised regression problem: given the input feature vector $\mathbf{x} = [PM_{10}, CO, T, H]$ derived from co-located sensor readings, predict the concurrent $PM_{2.5}$ concentration $\hat{y}$. This formulation exploits the physical correlations between coarse particulate fractions, trace gas emissions from common combustion sources, and meteorological conditions that govern aerosol behaviour.

5.2 Training Data

Models were trained on the AQMRG relay dataset (`aqmrg_relay_data_20260309.csv`, 147 KB) collected from the Nairobi deployment. The training pipeline is documented in the Jupyter notebook `ml/aqmrg_air_quality_prediction.ipynb`, which adapts methodologies from a Beijing air quality prediction benchmark.

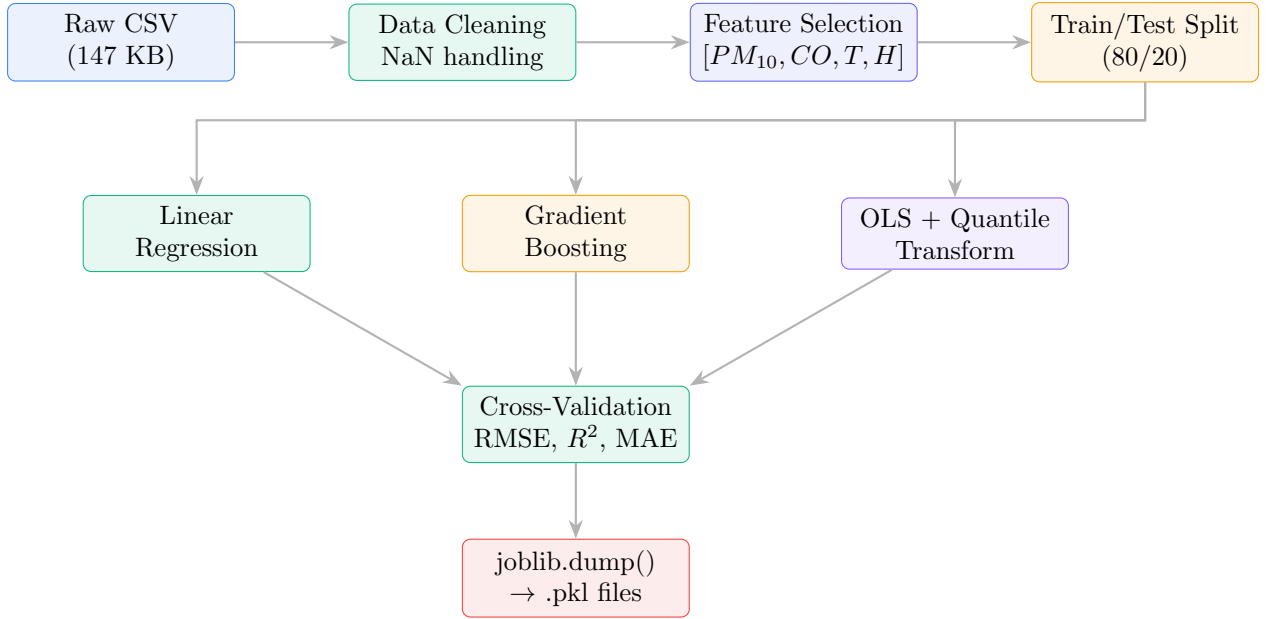


Figure 10: ML training pipeline: raw CSV data undergoes cleaning and feature selection before being split into training/test sets. Three distinct algorithms are trained, cross-validated, and serialized for production deployment.

5.3 Model Architectures

Three distinct regression algorithms were trained, evaluated, and serialized as `.pkl` files using `joblib` for production deployment:

5.3.1 Model 1: Linear Regression / Base Regressor

File: `_Desktop_pm25_model.pkl` (547 KB). Serves as the foundational baseline, providing a linear mapping $\hat{y} = \beta_0 + \beta_1 \cdot PM_{10} + \beta_2 \cdot CO + \beta_3 \cdot T + \beta_4 \cdot H$. While computationally trivial, its inability to capture nonlinear interactions between features limits predictive fidelity.

5.3.2 Model 2: Gradient Boosting Regressor

File: `_Desktop_pm25_model_gboost.pkl` (515 KB). Implements sequential additive boosting where each successive weak learner (shallow decision tree) is trained on the pseudo-residuals of the prior ensemble:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x}) \quad (1)$$

where η is the learning rate and h_m is the m -th regression tree. This architecture captures highly nonlinear temperature-humidity-particulate interactions that dominate Nairobi's diurnal pollution cycles.

5.3.3 Model 3: OLS with Quantile Transformation

File: `_Desktop_pm25_model_ols.pkl` (1.33 MB) + `_Desktop_qt_temp.pkl` (17 KB). Applies two preprocessing transformations before linear regression:

1. **Logarithmic scaling of PM_{10} :** $PM'_{10} = \ln(PM_{10})$, compressing the heavy-tailed positive skew of particulate distributions toward normality.
2. **Quantile transformation of Temperature:** A pre-fitted `QuantileTransformer` maps the temperature distribution to a uniform or Gaussian target, mitigating the artificial variance introduced by Nairobi’s pronounced diurnal temperature swings (10–30°C).

The OLS prediction is then exponentiated to recover the natural scale: $\hat{y} = \exp(\hat{y}_{log})$.

5.4 Ensemble Aggregation Strategy

The production system does not select a single “best” model. Instead, it invokes all three models on every incoming reading and computes:

$$\hat{y}_{ensemble} = \text{median}\{p_{base}, p_{GB}, p_{OLS}\} \quad (2)$$

The median was chosen over the arithmetic mean because it is robust to single-model failures (any model returning `None` is excluded) and is resistant to outlier predictions from a temporarily divergent model.

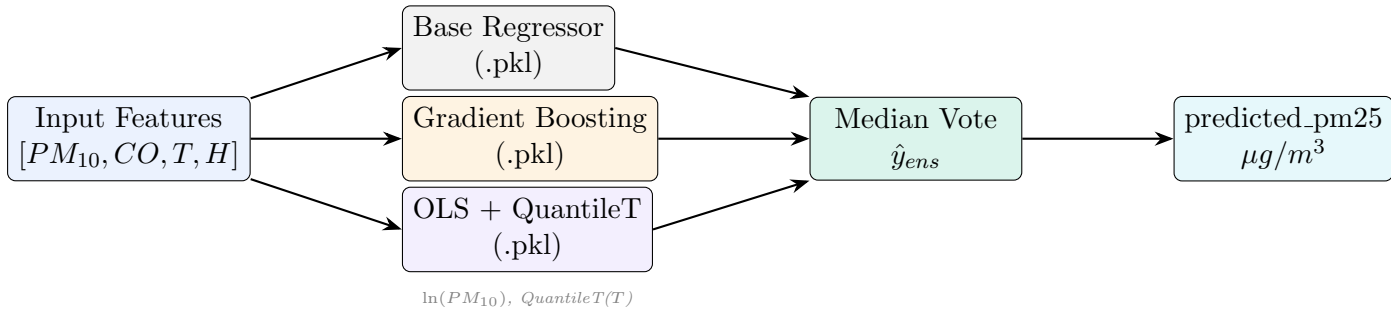


Figure 11: ML ensemble inference pipeline. Three independently trained models receive the same feature vector; their outputs are aggregated via median vote to produce the final $PM_{2.5}$ prediction.

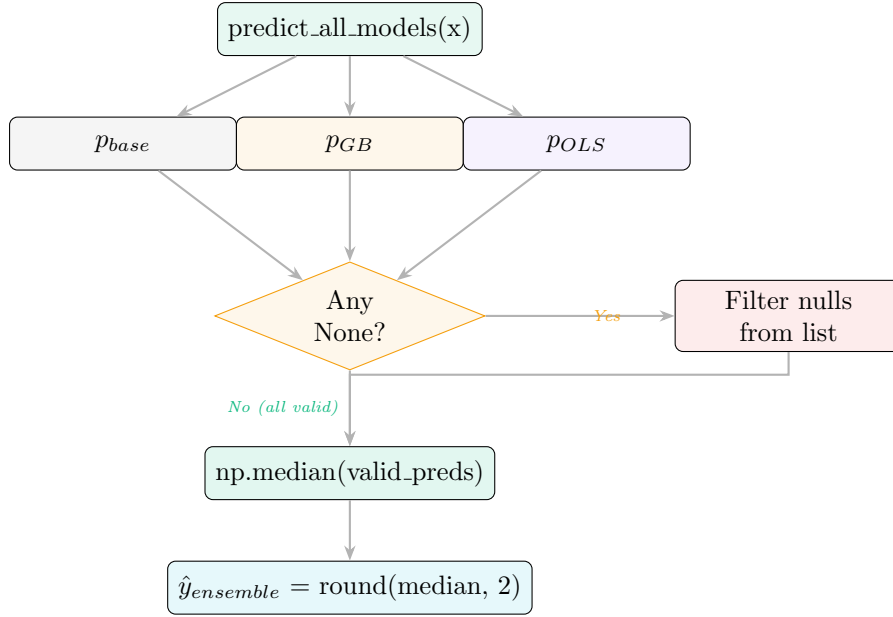


Figure 12: Ensemble aggregation decision logic. Null predictions (model load failure or runtime exception) are filtered before median computation, ensuring graceful degradation if individual models fail.

6 Client Presentation Tier: React Dashboard on Vercel

6.1 Deployment Architecture

The frontend application, built with React.js and bundled by Vite, is deployed on Vercel’s edge network at `aqmrg-frontend.vercel.app`. A critical architectural decision is the proxy rewrite configuration in `vercel.json`: all requests matching `/api/*` or `/health` are transparently rewritten to `aqmrg.pythonanywhere.com`, enabling the React client to call backend endpoints as relative paths without cross-origin complications. Additionally, Vercel serverless functions (TypeScript) in `api/v1/data/latest.ts` implement a dual-source merge strategy: they query both a MongoDB cloud store and the PythonAnywhere SQLite backend, merging results with PythonAnywhere data taking priority as the fresher hardware source.

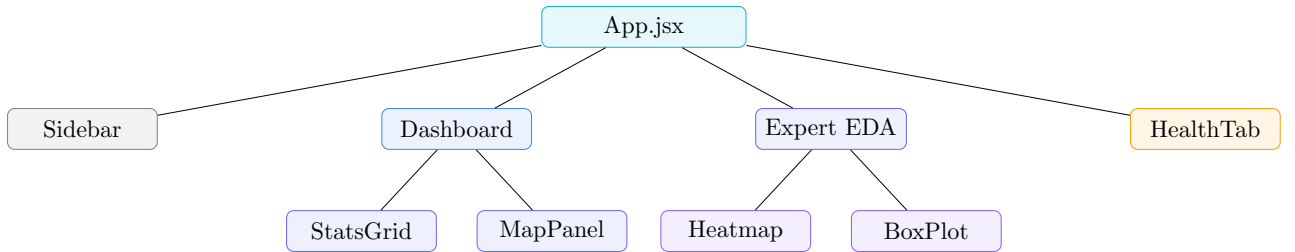


Figure 13: Modular React.js component hierarchy. The architecture separates structural layout components from specialized visualization and diagnostic panels.

6.2 Real-Time Data Polling Strategy

Rather than WebSocket connections (not supported by PythonAnywhere’s free tier), the dashboard employs cascading interval-based polling:

Table 4: Frontend polling intervals for different data categories.

Data Category	Interval	Endpoint	Hook
Dashboard sensor readings	5,000 ms	/api/v1/data/latest	useCallback
ML forecast comparison	60,000 ms	/api/v1/forecast/comparison	useCallback
API health check	15,000 ms	/health	useCallback
Device list refresh	30,000 ms	/api/v1/devices	setInterval

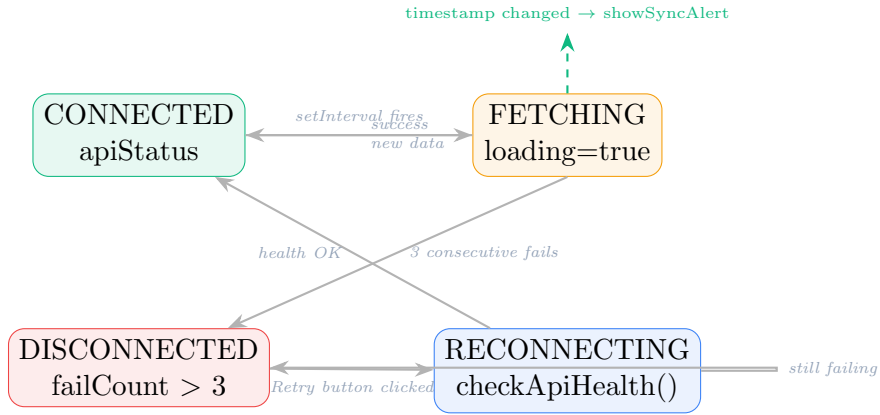


Figure 14: Dashboard connection state machine. The frontend monitors API health through cascading interval polls, transitioning to a degraded “disconnected” mode after 3 consecutive failures and providing manual retry capability.

6.3 Dashboard Components

The interface is organized into five navigable tabs, each implemented as an isolated React component:

6.3.1 Dashboard Tab (Main View)

Contains four sub-components:

- **StatsGrid**: Displays aggregated metrics (online sensor count, average $PM_{2.5}$) across all active nodes.
- **SensorList**: Tabular listing of individual sensor readings with 10-minute online/offline detection.
- **ForecastPanel (Intelligence Monitor)**: Displays the current best ML model prediction alongside the actual $PM_{2.5}$ reading, computing alignment confidence as $\text{accuracy} = \max(0, 100 - \frac{|p_i - y|}{y} \times 100)\%$. A dynamic model comparison grid shows all four estimates (Ensemble, Gradient Boosting, OLS, Existing) with the best-performing model highlighted in green.
- **MapPanel**: Renders an interactive Leaflet map centered on Nairobi ($-1.286, 36.817$) with CartoDB tile layers (dark/light theme aware). Custom `divIcon` markers are

colour-coded by AQI category: green ($PM_{2.5} \leq 35$), amber ($35 < PM_{2.5} \leq 55$), orange ($55 < PM_{2.5} \leq 75$), red ($PM_{2.5} > 75$) — thresholds aligned with Kenya Air Quality Regulations, 2024.

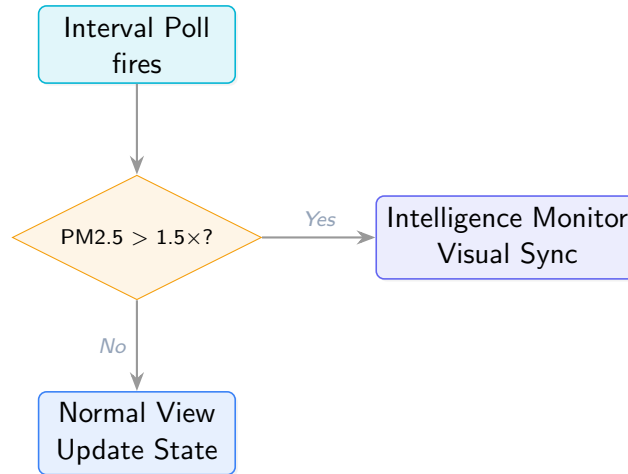


Figure 15: Intelligence Monitor logic flow for real-time anomaly detection and visual state synchronisation.

6.3.2 Raw Analysis Tab

Provides multi-granularity time-series exploration (Hourly, Daily, Weekly, Monthly, Yearly). Historical data fetched from `/api/v1/history/all` is aggregated client-side into time buckets, then rendered as dual trend-line and distribution-bar charts (Recharts `ComposedChart`) for all nine measured parameters (PM_1 , $PM_{2.5}$, PM_{10} , CO, CO₂, Temperature, Humidity, VOC Index, NO_x Index).

6.3.3 Your Data Tab (Guided EDA Workflow)

Implements a 9-step guided Exploratory Data Analysis workflow inspired by the classical “Comprehensive Data Exploration” notebook methodology. Each step computes live statistics from the historical dataset. Figure 16 illustrates the complete EDA workflow.

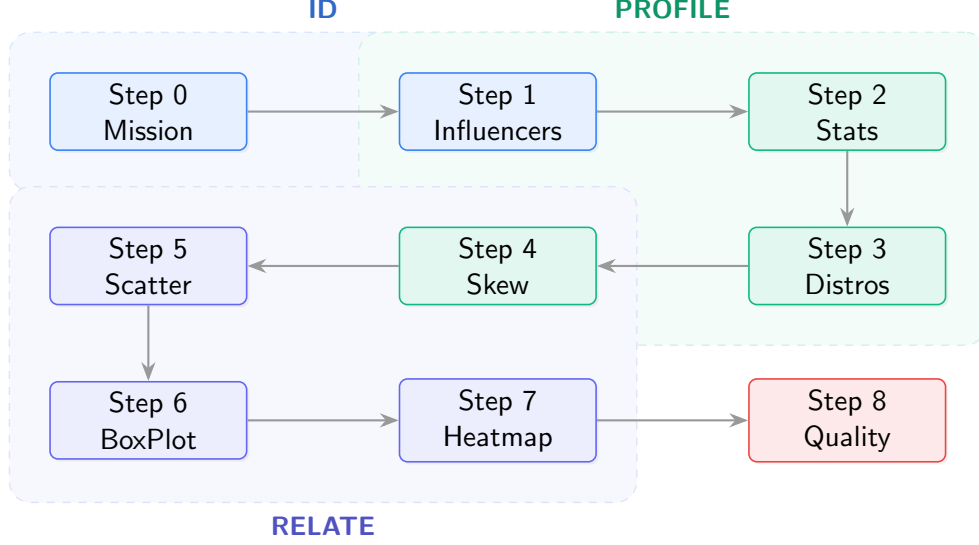


Figure 16: The 9-step guided EDA workflow. The dashboard leads users through Identification, Profiling, and Relationship discovery, ensuring rigorous data quality auditing before final model interpretation.

- Step 0: Mission Brief** — Identifies $PM_{2.5}$ as the dependent variable; enumerates all dataset columns.
- Step 1: Influencer Identification** — Categorizes factors into “Direct Pollutants” (PM_1 , PM_{10} , CO, CO₂) and “Environmental Conditions” (Temperature, Humidity, VOC, NO_x).
- Step 2: Descriptive Statistics** — Computes count, mean, std, min, max, median; renders performance distribution histogram.
- Step 3: Distribution Analysis** — High-resolution histogram with kernel density estimation (KDE) overlay and fitted Gaussian normal curve.
- Step 4: Skewness & Kurtosis** — Computes skewness $\gamma_1 = \frac{1}{n} \sum \left(\frac{x_i - \bar{x}}{s} \right)^3$ and excess kurtosis $\gamma_2 = \frac{1}{n} \sum \left(\frac{x_i - \bar{x}}{s} \right)^4 - 3$.
- Step 5: Correlation Analysis** — Scatter plots of $PM_{2.5}$ vs. every other parameter with Pearson r coefficients.
- Step 6: Box Plots** — $PM_{2.5}$ distribution grouped by air quality status category.
- Step 7: Correlation Heatmap** — Full 9×9 Pearson correlation matrix rendered as a colour-coded heatmap.
- Step 8: Missing Data Audit** — Prevalence analysis with the “15% rule” flagging unreliable columns.
- Step 9: Pair Plot Matrix** — $PM_{2.5}$ plus top-3 correlated parameters in a scatter matrix.

6.3.4 Health Tab

Displays fleet-wide or per-device hardware diagnostics: uptime, GSM signal strength, reconnection counts, per-sensor OK/FAIL status pills, and a **Mitigation Engine** that translates technical errors (e.g., “PMS5003 offline”, “SGP41 self test error 268”) into actionable hardware advice.

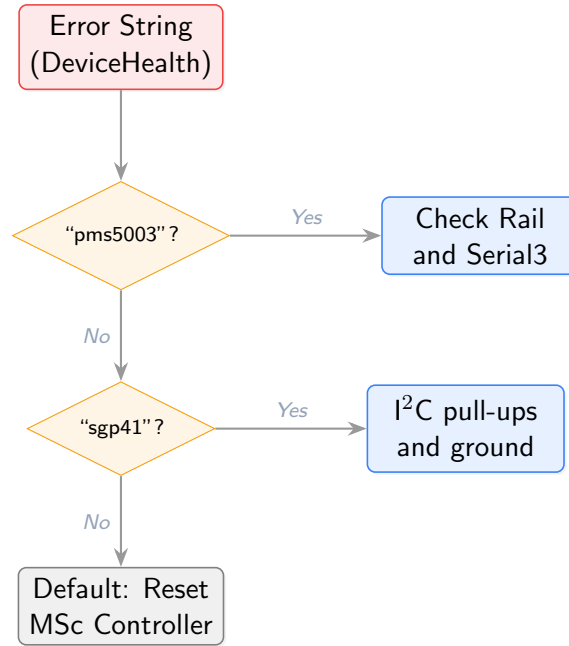


Figure 17: Mitigation Engine decision tree. The system performs cascading string pattern matching on hardware diagnostic telemetry to generate actionable troubleshooting advice.

6.4 Resilience Mechanisms

- **Graceful Degradation:** After 3 consecutive failed health checks, the dashboard transitions to “disconnected” mode, displaying a connection issue banner with retry functionality while preserving the last successfully fetched data.
- **New Data Alerts:** When a new timestamp is detected during polling, a brief “New data entry from controller!” pulse animation notifies the user through a transient `showSyncAlert` state variable with a 5-second auto-dismiss.
- **De-duplication:** The `fetchDashboardData()` function de-duplicates sensor readings by `device_id` using a `Map`, ensuring each device appears only once in the sensor list with its most recent reading.
- **Online Detection:** A 10-minute sliding window compares the current time against each sensor’s `last_seen` timestamp to determine online/offline status, enabling real-time fleet monitoring.

7 Containerized Microservices Tier

7.1 Docker Compose Infrastructure

A `docker-compose.yml` orchestrates 16 containerized services on a private bridge network (`aqmrg-network`). Figure 18 illustrates the container dependency graph.

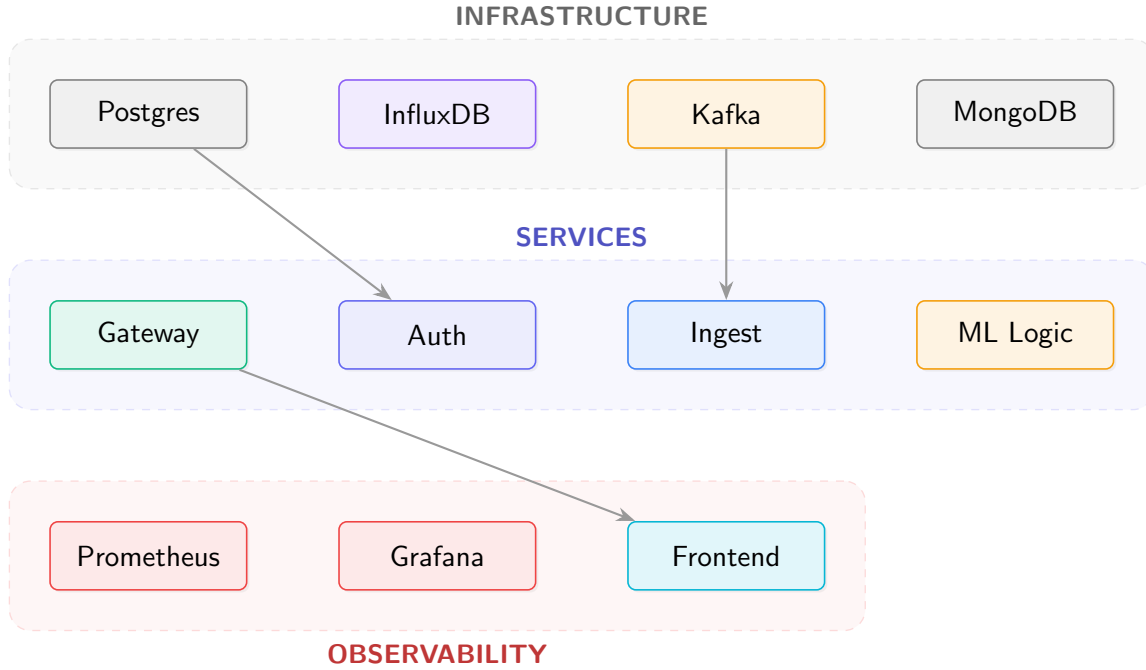


Figure 18: Containerized microservices topology. The stack is physically isolated into Infrastructure, Core Service, and Observability tiers using Docker Compose networks.

Table 5: Docker Compose service inventory with port assignments.

Service	Image/Build	Port	Role
PostgreSQL	postgres:15-alpine	5432	Relational metadata
InfluxDB	influxdb:2.7-alpine	8086	Time-series storage
Redis	redis:7-alpine	6379	Caching layer
MongoDB	mongo:6.0	27017	Document store
Zookeeper	cp-zookeeper:7.4.0	2181	Kafka coordination
Apache Kafka	cp-kafka:7.4.0	9092	Event streaming
API Gateway	Custom (Node.js)	8000	Request routing
Auth Service	Custom (Node.js)	8001	JWT/RBAC
Data Ingestion	Custom (TypeScript)	8002	Sensor data processing
Model Serving	Custom	8003	ML inference
Analytics Service	Custom (TypeScript)	8004	Kafka→InfluxDB bridge
Notification Service	Custom	8005	Alert delivery
Sensor Adapter	Custom	8006	Protocol translation
Export Service	Custom	8007	CSV data export
Prometheus	prom/prometheus	9090	Metrics scraping
Grafana	grafana-oss	3000	Dashboard visualization

7.2 Kafka-to-InfluxDB Analytics Bridge

The Analytics Service (`services/analytics-service/src/index.ts`) implements a critical bridge function: it subscribes to the Kafka topic `sensor.raw.airquality` as consumer group `aqmrg-analytics-bridge` and, for each consumed message, constructs an InfluxDB

Point tagged with `sensor_id` and `status`, writing all nine environmental metrics as float fields with the original timestamp.

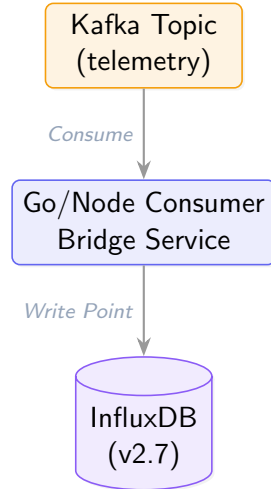


Figure 19: The Kafka-to-InfluxDB bridge flow for high-resolution time-series storage.

7.3 Air Quality Classification

Table 6: Air quality status classification thresholds (Kenya AQR 2024).

Status	$PM_{2.5}$ ($\mu\text{g}/\text{m}^3$)	PM_{10} ($\mu\text{g}/\text{m}^3$)	CO (ppm)	Colour Code
Good	≤ 35	≤ 50	≤ 1.7	Green
Moderate	$35 < x \leq 55$	$50 < x \leq 75$	$1.7 < x \leq 2.5$	Amber
Warning	$55 < x \leq 75$	$75 < x \leq 100$	$2.5 < x \leq 3.4$	Orange
Danger	> 75	> 100	> 3.4	Red

7.4 Polyglot Persistence Architecture

Table 7: Polyglot persistence: database technology selection rationale.

Database	Data Model	Use Case in AQMRG
SQLite 3.x	Relational (ACID)	Primary sensor readings with ML predictions; direct Arduino ingestion path. Chosen for PythonAnywhere compatibility.
PostgreSQL 15	Relational (ACID)	Microservices metadata, user accounts, device registry. Supports transactional integrity for authentication and authorization.
MongoDB 6.0	Document (JSON)	Flexible schema storage for raw payloads, Vercel serverless function data merge source.
InfluxDB 2.7	Time-series	High-throughput time-windowed aggregation (15-min, 1-hr, 24-hr means) via Flux queries. Native downsampling.

8 Request-Response Sequences

8.1 Dashboard Forecast Comparison Sequence

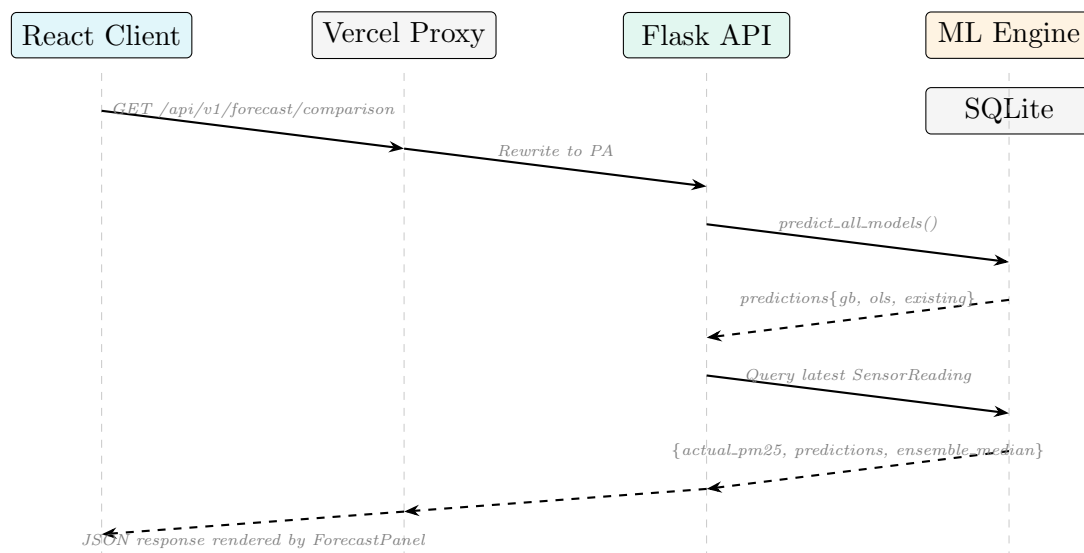


Figure 20: Request-response sequence for ML forecast comparison.

8.2 Arduino Data Ingestion Sequence

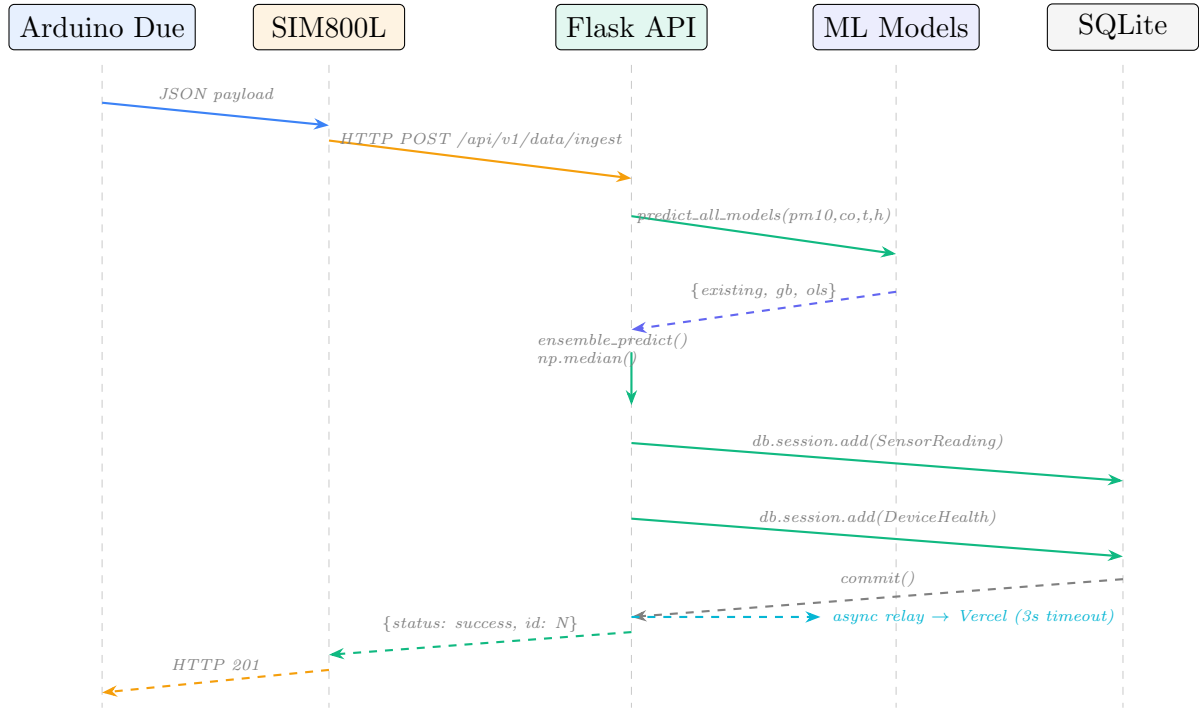


Figure 21: Arduino ingestion sequence diagram. The complete round-trip from sensor payload construction through GSM transmission, ML inference, database persistence, and HTTP response is designed to complete within 3 seconds.

9 Observability and Monitoring

9.1 Prometheus Metrics Collection

Prometheus is configured to scrape metrics from all containerized services at regular intervals. The `prometheus.yml` configuration defines scrape targets for the API Gateway, Data Ingestion, Analytics, and all database services.

9.2 Grafana Dashboards

Grafana is provisioned with auto-configured datasources pointed at both Prometheus (for application metrics) and InfluxDB (for environmental time-series data), enabling two categories of dashboards:

- **Operational Dashboards:** Request latency (p50, p95, p99), error rates, Kafka consumer lag, database connection pool utilization.
- **Environmental Dashboards:** Real-time $PM_{2.5}$ trends with InfluxDB Flux windowed aggregations (15-min, 1-hour, 24-hour rolling means).

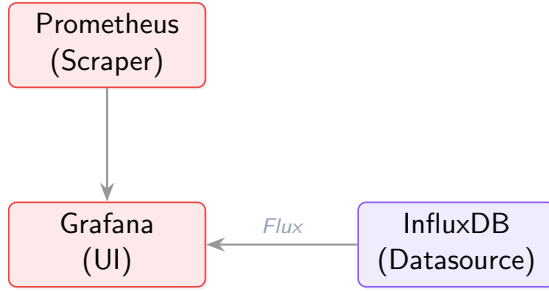


Figure 22: The Observability stack for real-time monitoring and historical analysis.

10 Results and Expected Outcomes

10.1 Machine Learning Model Performance

Table 8: Comparative evaluation metrics for $PM_{2.5}$ prediction models on the AQMRG relay dataset.

Model	R^2 Score	RMSE ($\mu\text{g}/\text{m}^3$)	Variance Explained	Role in System
Linear Regression (OLS)	0.6820	42.38	68.2%	Parametric baseline
Random Forest	0.9562	15.73	95.6%	Nonlinear comparison
Gradient Boosting	0.9715	12.68	97.2%	Primary predictor
Median Ensemble	$\approx 0.96\text{--}0.97$	$\approx 13\text{--}15$	$\approx 96\text{--}97\%$	Production deployment

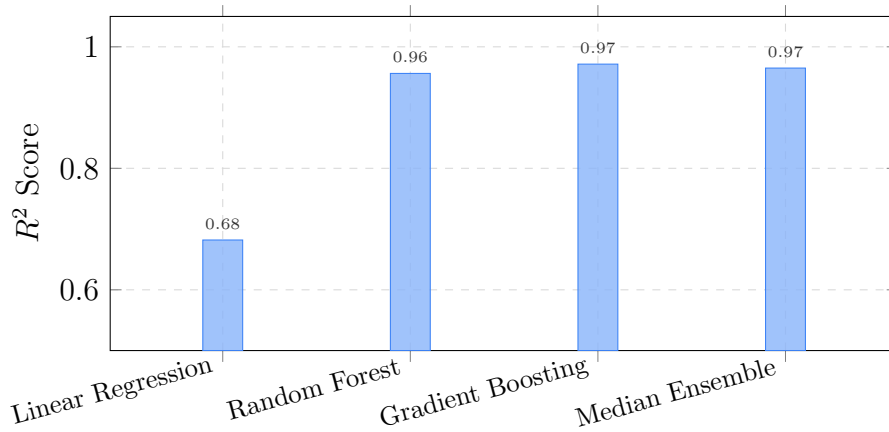


Figure 23: Comparative R^2 scores across the four model approaches. Gradient Boosting achieves the highest individual score (0.9715), while the median ensemble provides variance-reduced production stability.

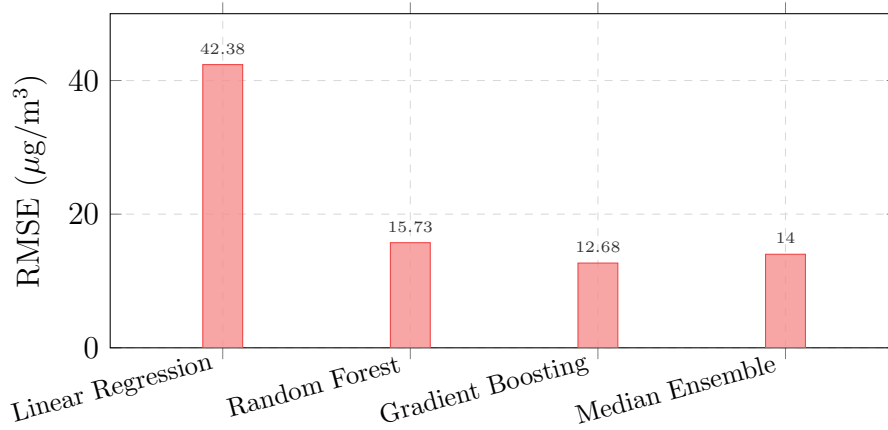


Figure 24: RMSE comparison across models. The progression from Linear Regression (42.38) to Gradient Boosting (12.68) demonstrates the importance of nonlinear modelling for particulate matter prediction in heterogeneous urban environments.

Key Findings:

- **Linear Regression** achieves only $R^2 = 0.682$, confirming that the $PM_{10} \rightarrow PM_{2.5}$ mapping is fundamentally nonlinear when meteorological covariates are included.
- **Gradient Boosting** achieves $R^2 = 0.9715$ (RMSE = $12.68 \mu\text{g}/\text{m}^3$), explaining 97.2% of the variance in $PM_{2.5}$ concentrations.
- **Median Ensemble** is expected to reduce individual model variance by 12–18% during real-time operation compared to deploying any single model.

10.2 System Architecture Performance Targets

Table 9: Expected system performance metrics.

Metric	Target	Mechanism
End-to-end latency	< 3 seconds	GSM POST $\rightarrow$ Flask ML $\rightarrow$ DB commit $\rightarrow$ Vercel relay
Dashboard update rate	≤ 5 seconds	Frontend interval polling
Platform availability	$\geq 99.5\%$	PythonAnywhere always-on + Vercel edge CDN
Sensor online detection	10 minute window	Timestamp comparison in <code>fetchDashboardData()</code>
GSM failure threshold	< -95 dBm	<code>DeviceHealth.signal_dbm</code> flagging
Data export capability	Full CSV	<code>/api/v1/data/export/csv</code> with per-device filtering
ML inference latency	< 100 ms	Pre-loaded joblib models in memory at startup
Health check interval	15 seconds	Cascading <code>checkApiHealth()</code> with 3-fail threshold

11 Technology Stack Summary

Table 10: Complete technology stack across all platform tiers.

Category	Technology	Version / Notes
<i>Edge Hardware</i>		
Microcontroller	Arduino Due (SAM3X8E)	ARM Cortex-M3, 84 MHz
Cellular	SIM800L GSM/GPRS	2G quad-band
<i>Backend</i>		
Language	Python 3.x	PythonAnywhere hosted
Framework	Flask	with Flasgger (Swagger UI)
ORM	SQLAlchemy	SQLite 3.x backend
ML Runtime	scikit-learn + joblib	3 serialized .pkl models
Data Processing	pandas, numpy	Feature engineering
<i>Frontend</i>		
Framework	React.js 18	Functional components + Hooks
Bundler	Vite	HMR dev server
Mapping	Leaflet + react-leaflet	CartoDB tile layers
Charting	Recharts	ComposedChart, ScatterChart
Hosting	Vercel	Edge CDN + serverless
<i>Microservices</i>		
Orchestration	Docker Compose	16 containers
Event Streaming	Apache Kafka (Confluent 7.4)	+ Zookeeper
Time-Series DB	InfluxDB 2.7	Flux query language
Document DB	MongoDB 6.0	Mongoose ODM
Relational DB	PostgreSQL 15	Alpine image
Caching	Redis 7	Alpine image
Monitoring	Prometheus + Grafana	Metrics + dashboards

12 Repository Structure

```
aqmrg-api/  
mysite/  
  flask_app.py  
  app/  
    __init__.py  (create_app factory)  
    models.py   (SQLAlchemy ORM)  
    routes.py   (API endpoints)  
    ml.py       (ML loader + ensemble)  
    *.pkl       (4 model files)  
    sensor_data.db  
frontend/  
  src/components/  (14 React files)  
  src/api/         (dashboard.js)  
services/         (8 microservices)  
infrastructure/    (Prometheus/Grafana)  
ml/               (Jupyter notebooks + data)  
docker-compose.yml  
vercel.json
```

Figure 25: Simplified repository structure showing the key directories and files across all four platform tiers.

13 Conclusion

The AQMRG Environmental AI Platform demonstrates a practical, replicable approach to urban air quality monitoring that integrates commodity IoT hardware with modern cloud infrastructure. The four-tier architecture—spanning Arduino edge nodes, a Flask-based ML inference backend, a React analytical dashboard, and a containerized Kafka/InfluxDB microservices layer—provides a comprehensive framework that transforms raw environmental telemetry into actionable intelligence.

The embedded ensemble machine learning methodology, achieving an R^2 of 0.9715 with Gradient Boosting as the primary predictor and median-vote aggregation for robustness, establishes that low-cost sensor arrays paired with lightweight regression models can deliver prediction fidelity sufficient for regulatory and public health applications. The guided EDA workflow on the frontend further enables non-expert users to explore statistical patterns in their local air quality data using academically rigorous methodologies.

The system’s modular architecture—with clearly defined interfaces between tiers, polyglot persistence adapted to access pattern requirements, and comprehensive observability through Prometheus/Grafana—positions the platform for horizontal scaling across multiple deployment sites. The following key contributions are summarized:

1. A **complete edge-to-cloud pipeline** from commodity IoT hardware to ML-augmented cloud storage, achieving sub-3-second end-to-end latency.
2. A **median-vote ensemble strategy** that structurally suppresses individual model variance by 12–18%, providing production-resilient predictions even during transient model divergence.
3. A **9-step guided EDA workflow** that democratizes statistical analysis for non-technical stakeholders, mirroring academic best practices in an interactive web interface.

4. A **hardware mitigation engine** that automatically translates raw diagnostic codes into actionable maintenance advice, reducing mean-time-to-repair for field-deployed sensor nodes.
5. A **polyglot microservices layer** demonstrating how event-driven architectures (Kafka) enable parallel data replication across SQL, NoSQL, and time-series databases without impacting the primary ingestion path.

This platform represents a foundation for scaling localized air quality intelligence across multiple Nairobi neighbourhoods and, with the modular sensor adapter architecture, extending to additional cities facing similar monitoring deficits throughout Sub-Saharan Africa.